

Регулярные выражения



ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Определение регулярных выражений. **3**

Символы литералов **4**

Классы символов **5**

Повторение **5**

Альтернативы, группировка и ссылки **7**

Указание позиции соответствия **8**

Флаги **10**

Методы класса String для поиска по шаблону **10**

Метод replace(). 11

Метод match(). 11

Объект RegExp **12**

Свойства RegExp. 13

Методы RegExp. 13

Литература и другие источники *15*

Вопросы для самопроверки *15*

ПЛАН ЛЕКЦИИ

1. Определение регулярных выражений;
2. Символы литералов;
3. Классы символов;
4. Повторение;
5. Альтернативы, группировки и ссылки;
6. Указание позиции соответствия;
7. Флаги;
8. Методы класса String для поиска по шаблону;
9. Объект RegExp.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Регулярное выражение - это объект, описывающий символьный шаблон.

Класс RegExp в JavaScript представляет регулярные выражения, а объекты классов String и RegExp определяют методы, использующие регулярные выражения для выполнения поиска по шаблону и операций поиска в тексте с заменой.

Грамматика регулярных выражений в языке JavaScript содержит достаточно полное подмножество синтаксиса регулярных выражений, используемого в языке Perl 5, поэтому, если вы имеете опыт работы с языком Perl, то вы без труда сможете описывать шаблоны в программах на языке JavaScript.

В число особенностей регулярных выражений языка Perl, которые не поддерживаются в ECMAScript, входят флаги s (однострочный режим) и x (расширенный синтаксис); управляющие последовательности \a, \e, \l, \u, \L, \U, \E, \Q, \A, \Z, \z и \G и другие расширенные конструкции, начинающиеся с (?.

ОПРЕДЕЛЕНИЕ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ.

В JavaScript регулярные выражения представлены ***объектами RegExp***. Объекты RegExp могут быть созданы посредством конструктора RegExp(), но чаще они создаются с помощью специального синтаксиса литералов. Так же как ***строковые литералы*** задаются в виде символов, заключенных в кавычки, литералы регулярных выражений задаются в виде символов, заключенных в пару символов слэша (/). Таким образом, JavaScript-код может содержать строки, похожие на эту:

```
var pattern = /s$/;
```

Эта строка создает новый объект RegExp и присваивает его переменной pattern. Данный объект RegExp ищет любые строки, заканчивающиеся символом "s". Это же регулярное выражение может быть определено с помощью конструктора RegExp():

```
var pattern = new RegExp("s$");
```

Спецификация шаблона регулярного выражения состоит из последовательности символов. Большая часть символов, включая все алфавитно-цифровые, буквально описывают символы, которые должны присутствовать. То есть регулярное выражение /java/ совпадает со всеми строками, содержащими подстроку «java».

Другие символы в регулярных выражениях не предназначены для поиска их точных эквивалентов, а имеют особое значение. Например, регулярное выражение /s$/ содержит два символа. Первый символ s обозначает поиск буквального символа. Второй, $ - это специальный метасимвол, обозначающий

конец строки. Таким образом, это регулярное выражение соответствует любой строке, заканчивающейся символом s.

В следующих разделах описаны различные символы и метасимволы, используемые в регулярных выражениях в языке JavaScript.

Символы литералов

Как отмечалось ранее, все алфавитные символы и цифры в регулярных выражениях соответствуют сами себе. Синтаксис регулярных выражений в JavaScript также поддерживает возможность указывать некоторые неалфавитные символы с помощью управляющих последовательностей, начинающихся с символа обратного слэша (\). Например, последовательность \n соответствует символу перевода строки. Эти символы перечислены в таблице ниже:

Символы в регулярных выражениях JavaScript

Символ	Соответствие
\0	Символ NUL (\u0000)
\t	Табуляция (\u0009)
\n	Перевод строки (\u000A)
\v	Вертикальная табуляция (\u000B)
\f	Перевод страницы (\u000C)
\r	Возврат каретки (\u000D)
\xnn	Символ из набора Latin, задаваемый шестнадцатеричным числом nn; например, \x0A - это то же самое, что \n
\uxxxx	Unicode-символ, заданный шестнадцатеричным числом xxxx; например, \u0009 - это то же самое, что \t
\cX	Управляющий символ "X", например, последовательность \cJ эквивалентна символу перевода строки \n

Некоторые знаки препинания имеют в регулярных выражениях особый смысл:

^ $. * + ? = ! : | \ / ( ) [ ] { } -

Значение этих символов раскрывается в последующих разделах. Некоторые из них имеют специальный смысл только в определенных контекстах регулярных выражений, а в других контекстах трактуются буквально. Однако, как правило, чтобы включить какой-либо из этих символов в регулярное выражение буквально, необходимо поместить перед ним символ обратного слэша. Другие символы, такие как кавычки и @, не имеют специального значения и просто соответствуют в регулярных выражениях самим себе.

Если вы не можете точно вспомнить, каким из символов должен предшествовать символ \, можете спокойно помещать обратный слэш перед любым из символов. Однако имейте в виду, что многие буквы и цифры вместе с символом слэша обретают специальное значение, поэтому тем буквам и цифрам, которые вы ищете буквально, не должен предшествовать символ \. Чтобы включить в регулярное выражение сам символ обратного слэша, перед ним, очевидно, следует поместить другой символ обратного слэша. Например, следующее регулярное выражение соответствует любой строке, содержащей символ обратного слэша: /\\/.

Классы символов

Отдельные символы литералов могут объединяться в ***классы символов*** путем помещения их в квадратные скобки. ***Класс символов*** соответствует любому символу, содержащемуся в этом классе. Следовательно, регулярное выражение /[abc]/ соответствует одному из символов a, b или c.

Могут также определяться классы символов с отрицанием, соответствующие любому символу, кроме тех, которые указаны в скобках. Класс символов с отрицанием задается символом ^ в качестве первого символа, следующего за левой скобкой. Регулярное выражение /[^abc]/ соответствует любому символу, отличному от a, b или c. В классах символов диапазон символов может задаваться при помощи дефиса. Поиск всех символов латинского алфавита в нижнем регистре осуществляется посредством выражения /[a-z]/, а любую букву или цифру из набора символов Latin можно найти при помощи выражения /[a-zA-Z0-9]/.

Некоторые классы символов используются особенно часто, поэтому синтаксис регулярных выражений в JavaScript включает специальные символы и управляющие (escape) последовательности для их обозначения. Так, \s соответствует символам пробела, табуляции и любым пробельным символам из набора Unicode, а \S - любым символам, не являющимся пробельными символами из набора Unicode.

В таблице ниже приводится перечень этих спецсимволов и синтаксиса классов символов. (Обратите внимание, что некоторые из управляющих последовательностей классов символов соответствуют только ASCII-символам и не расширены для работы с Unicode-символами. Можно явно определить собственные классы Unicode-символов, например, выражение /[\u0400-\u04FF]/ соответствует любому символу кириллицы.)

Классы символов регулярных выражений JavaScript

Символ	***Соответствие***
[...]	Любой из символов, указанных в скобках
[^...]	Любой из символов, не указанных в скобках
.	Любой символ, кроме перевода строки или другого разделителя Unicode-строки
\w	Любой текстовый ASCII-символ. Эквивалентно [a-zA-Z0-9_]
\W	Любой символ, не являющийся текстовым ASCII-символом. Эквивалентно [^a-zA-Z0-9_]
\s	Любой пробельный символ из набора Unicode
\S	Любой непробельный символ из набора Unicode. Обратите внимание, что символы \w и \S - это не одно и то же
\d	Любые ASCII-цифры. Эквивалентно [0-9]
\D	Любой символ, отличный от ASCII-цифр. Эквивалентно [^0-9]
[\b]	Литерал символа «забой»

Обратите внимание, что управляющие последовательности специальных символов классов могут находиться в квадратных скобках. \s соответствует любому пробельному символу, а \d соответствует любой цифре, следовательно, /[\s\d]/ соответствует любому пробельному символу или цифре.

Повторение

Имея знания синтаксиса регулярных выражений, полученные к настоящему моменту, мы можем описать число из двух цифр как /\d\d/ или из четырех цифр как /\d\d\d\d/, но не сможем, например, описать число, состоящее из любого количества цифр, или строку из трех букв, за которыми следует

необязательная цифра. Эти более сложные шаблоны используют синтаксис регулярных выражений, указывающий, сколько раз может повторяться данный элемент регулярного выражения.

Символы, обозначающие повторение, всегда следуют за шаблоном, к которому они применяются. Некоторые виды повторений используются довольно часто, и для обозначения этих случаев имеются специальные символы. Например, + соответствует одному или нескольким экземплярам предыдущего шаблона. В следующей таблице приведена сводка синтаксиса повторений:

Символы повторения в регулярных выражениях JavaScript

Символ	***Значение***
{n,m}	Соответствует предшествующему шаблону, повторенному не менее n и не более m раз
{n,}	Соответствует предшествующему шаблону, повторенному n или более раз
{n}	Соответствует в точности n экземплярам предшествующего шаблона
?	Соответствует нулю или одному экземпляру предшествующего шаблона; предшествующий шаблон является необязательным. Эквивалентно {0,1}
+	Соответствует одному или более экземплярам предшествующего шаблона. Эквивалентно {1,}
*	Соответствует нулю или более экземплярам предшествующего шаблона. Эквивалентно {0,}

Следующие строки демонстрируют несколько примеров:

```
var pattern = /\d{2,4}/;   // Соответствует числу, содержащему от двух до четырех цифр
pattern = /\w{3}\d?/;      // Соответствует в точности трем символам слова и одной необязательной цифре
pattern = /\s+java\s+/;    // Соответствует слову "java" с одним или более пробелами // до и после него
pattern = /[^(]*/;         // Соответствует нулю или более символам, отличным от открывающей круглой скобки
```

Будьте внимательны при использовании символов повторения * и ?. Они могут соответствовать отсутствию указанного перед ними шаблона и, следовательно, отсутствию символов. Например, регулярному выражению /a*/ соответствует строка "bbbb", поскольку в ней нет символа a.

Символы повторения, перечисленные в таблице, соответствуют максимально возможному количеству повторений, при котором обеспечивается поиск последующих частей регулярного выражения. Мы говорим, что это - «жадное» повторение. Имеется также возможность реализовать повторение, выполняемое «нежадным» способом. Достаточно указать после символа (или символов) повторения вопросительный знак: ??, +?, *? или даже {1,5}?.

Например, регулярное выражение /a+/ соответствует одному или более экземплярам буквы a. Примененное к строке «aaa», оно соответствует всем трем буквам. С другой стороны, выражение /a+?/ соответствует одному или более экземплярам буквы a и выбирает наименее возможное число символов. Примененный к той же строке, этот шаблон соответствует только первой букве a.

«Нежадное» повторение не всегда дает ожидаемый результат. Рассмотрим шаблон /a+b/, соответствующий одному или более символам a, за которыми следует символ b. Применительно к строке «aaab» ему соответствует вся строка.

Теперь проверим «нежадную» версию /a+?b/. Можно было бы подумать, что она должна соответствовать символу b, перед которым стоит только один символ a. В случае применения к той же строке «aaab» можно было бы ожидать, что она совпадет с единственным символом a и последним символом b. Однако на самом деле этому шаблону соответствует вся строка, как и в случае «жадной» версии. Дело в том, что поиск по шаблону регулярного выражения выполняется путем нахождения первой позиции в строке, начиная с которой соответствие становится возможным. Так как соответствие возможно, начиная с первого символа строки, более короткие соответствия, начинающиеся с последующих символов, даже не рассматриваются.

Альтернативы, группировка и ссылки

Грамматика регулярных выражений включает специальные символы определения ***альтернатив***, подвыражений группировки и ссылок на предыдущие подвыражения. Символ вертикальной черты | служит для разделения альтернатив. Например, /ab|cd|ef/ соответствует либо строке «ab», либо строке «cd», либо строке «ef», а шаблон /\d{3}|[a-z]{4}/ - либо трем цифрам, либо четырем строчным буквам.

Обратите внимание, что альтернативы обрабатываются слева направо до тех пор, пока не будет найдено соответствие. При обнаружении совпадения с левой альтернативой правая игнорируется, даже если можно добиться «лучшего» соответствия. Поэтому, когда к строке «ab» применяется шаблон /a|ab/, он будет соответствовать только первому символу.

Круглые скобки имеют в регулярных выражениях несколько значений. Одно из них - ***группировка*** отдельных элементов в одно подвыражение, так что элементы при использовании специальных символов |, *, +, ? и других рассматриваются как одно целое. Например, шаблон /java(script)?/ соответствует слову «java», за которым следует необязательное слово «script», а /(ab|cd)+|ef)/ соответствует либо строке «ef», либо одному или более повторений одной из строк «ab» или «cd».

Другим применением скобок в регулярных выражениях является определение ***подшаблонов*** внутри шаблона. Когда в целевой строке найдено совпадение с регулярным выражением, можно извлечь часть целевой строки, соответствующую любому конкретному подшаблону, заключенному в скобки.

Предположим, что требуется отыскать одну или более букв в нижнем регистре, за которыми следует одна или несколько цифр. Для этого можно воспользоваться шаблоном /[a-z]+\d+/. Но предположим также, что нам нужны только цифры в конце каждого соответствия. Если поместить эту часть шаблона в круглые скобки (/[a-z]+(\d+)/), то можно будет извлечь цифры из любых найденных нами соответствий. Как это делается, будет описано ниже.

С этим связано еще одно применение подвыражений в скобках, позволяющее ссылаться на подвыражения из предыдущей части того же регулярного выражения. Это достигается путем указания одной или нескольких цифр после символа \. Цифры ссылаются на позицию подвыражения в скобках внутри регулярного выражения. Например, \1 ссылается на первое подвыражение, а \3 - на третье. Обратите внимание, что подвыражения могут быть вложены одно в другое, поэтому при подсчете используется позиция левой скобки. Например, в следующем регулярном выражении ссылка на вложенное подвыражение ([Ss]cript) будет выглядеть как \2:

```
/([Jj]ava([Ss]cript)?)\sis\s(fun\w*)/
```

Ссылка на предыдущее подвыражение указывает не на шаблон этого подвыражения, а на найденный текст, соответствующий этому шаблону. Поэтому ссылки могут использоваться для наложения ограничения, выбирающего части строки, содержащие точно такие же символы. Например, следующее регулярное выражение соответствует нулю или более символам внутри одинарных или двойных кавычек.

Однако оно не требует, чтобы открывающие и закрывающие кавычки соответствовали друг другу (т.е. чтобы обе кавычки были одинарными или двойными):

```
/['"][^'"]*['"]/
```

Соответствия кавычек мы можем потребовать посредством такой ссылки:

```
/['"][^'"]*\1/
```

Здесь \1 соответствует совпадению с первым подвыражением. В этом примере ссылка налагает ограничение, требующее, чтобы закрывающая кавычка соответствовала открывающей. Это регулярное выражение не допускает присутствия одинарных кавычек внутри двойных, и наоборот.

Возможна также группировка элементов в регулярном выражении без создания нумерованной ссылки на эти элементы. Вместо простой группировки элементов между (и) начните группу с символов (?: и закончите ее символом). Рассмотрим, например, следующий шаблон:

```
/([Jj]ava(?:[Ss]cript)?)\sis\s(fun\w*)/
```

Здесь подвыражение (?:[Ss]cript) необходимо только для группировки, чтобы к группе мог быть применен символ повторения ?. Эти модифицированные скобки не создают ссылку, поэтому в данном регулярном выражении \2 ссылается на текст, соответствующий шаблону (fun\w*).

В следующей таблице приводится перечень операторов выбора из альтернатив, группировки и ссылки в регулярных выражениях:

Символы регулярных выражений выбора из альтернатив, группировки и ссылки JavaScript

Символ	***Значение***
|	Альтернатива. Соответствует либо подвыражению слева, либо подвыражению справа.
(...)	Группировка. Группирует элементы в единое целое, которое может использоваться с символами *, +, ?, | и т.п. Также запоминает символы, соответствующие этой группе для использования в последующих ссылках.
(?:...)	Только группировка. Группирует элементы в единое целое, но не запоминает символы, соответствующие этой группе.
\number	Соответствует тем же символам, которые были найдены при сопоставлении с группой с номером number. Группы - это подвыражения внутри скобок (возможно, вложенных). Номера группам присваиваются путем подсчета левых скобок слева направо. Группы, сформированные с помощью символов (?:, не нумеруются.

Указание позиции соответствия

Многие элементы регулярного выражения соответствуют одному символу в строке. Например, \s соответствует одному пробельному символу. Другие элементы регулярных выражений соответствуют позициям между символами, а не самим символам. Например, \b соответствует границе слова - границе между \w (текстовый ASCII-символ) и \W (нетекстовый символ) или границе между текстовым ASCII-символом и началом или концом строки.

Такие элементы, как \b, не определяют какие-либо символы, которые должны присутствовать в найденной строке, однако они определяют ***допустимые позиции для проверки соответствия***. Иногда эти элементы называются ***якорными элементами регулярных выражений***, потому что они закрепляют шаблон за определенной позицией в строке. Чаще других используются такие якорные элементы, как ^ и $, привязывающие шаблоны соответственно к началу и концу строки.

Например, слово «JavaScript», находящееся на отдельной строке, можно найти с помощью регулярного выражения /^JavaScript$/. Чтобы найти отдельное слово «Java» (а не префикс, например, в слове

«JavaScript»), можно попробовать применить шаблон /\sJava\s/, который требует наличия пробела до и после слова.

Но такое решение порождает две проблемы. Во-первых, оно найдет слово «Java», только если оно окружено пробелами с обеих сторон, и не сможет найти его в начале или в конце строки. Во-вторых, когда этот шаблон действительно найдет соответствие, возвращаемая им строка будет содержать ведущие и замыкающие пробелы, а это не совсем то, что нам нужно. Поэтому вместо шаблона, совпадающего с пробельными символами \s, мы воспользуемся шаблоном (или якорем), совпадающим с границами слова \b. Получится следующее выражение: /\bJava\b/.

Якорный элемент \B соответствует позиции, не являющейся границей слова. То есть шаблону /\B[Ss]cript/ будут соответствовать слова «JavaScript» и «postscript» и не будут соответствовать слова «script» или «Scripting».

В качестве якорных условий могут также выступать произвольные регулярные выражения. Если поместить выражение между символами (?= и), оно превратится в опережающую проверку на совпадение с последующими символами, требующую, чтобы эти символы соответствовали указанному шаблону, но не включались в строку соответствия.

Например, чтобы найти совпадение с названием распространенного языка программирования, за которым следует двоеточие, можно воспользоваться выражением /[Jj]ava([Ss]cript)?(?=\:)/. Этому шаблону соответствует слово «JavaScript» в строке «JavaScript: The Definitive Guide», но ему не будет соответствовать слово «Java» в строке «Java in a Nutshell», потому что за ним не следует двоеточие.

Если же ввести условие (?!, то это будет негативная опережающая проверка на последующие символы, требующая, чтобы следующие символы не соответствовали указанному шаблону. Например, шаблону /Java(?!Script)([A - Z]\w*)/ соответствует подстрока «Java», за которой следует заглавная буква и любое количество текстовых ASCII-символов при условии, что за подстрокой «Java» не следует подстрока «Script». Он совпадет со строкой «JavaBeans», но не совпадет со строкой «Javanese», совпадет со строкой «JavaScrip», но не совпадет со строками «JavaScript» или «JavaScripter».

В таблице ниже приводится перечень якорных символов регулярных выражений:

Символ	Значение
^	Соответствует началу строкового выражения или началу строки при многострочном поиске.
$	Соответствует концу строкового выражения или концу строки при многострочном поиске.
\b	Соответствует границе слова, т.е. соответствует позиции между символом \w и символом \W или между символом \w и началом или концом строки. (Однако обратите внимание, что [\b] соответствует символу забоя.)
\B	Соответствует позиции, не являющейся границей слов.
(?=p)	Позитивная опережающая проверка на последующие символы. Требует, чтобы последующие символы соответствовали шаблону p, но не включает эти символы в найденную строку.
(?!p)	Негативная опережающая проверка на последующие символы. Требует, чтобы следующие символы не соответствовали шаблону p.

Флаги

И еще один, последний элемент грамматики регулярных выражений. Флаги регулярных выражений задают высокоуровневые правила соответствия шаблонам. В отличие от остальной грамматики регулярных выражений, флаги указываются не между символами слэша, а после второго из них. В языке JavaScript поддерживается три флага.

Флаг i указывает, что поиск по шаблону должен быть нечувствителен к регистру символов, а флаг g - что поиск должен быть глобальным, т.е. должны быть найдены все соответствия в строке. Флаг m выполняет поиск по шаблону в многострочном режиме. Если строковое выражение, в котором выполняется поиск, содержит символы перевода строк, то в этом режиме якорные символы ^ и $, помимо того, что они соответствуют началу и концу всего строкового выражения, также соответствуют началу и концу каждой текстовой строки. Например, шаблону /java$/im соответствует как слово «java», так и «Java\nis fun».

Эти флаги могут объединяться в любые комбинации. Например, чтобы выполнить поиск первого вхождения слова «java» (или «Java», «JAVA» и т.д.) без учета регистра символов, можно воспользоваться нечувствительным к регистру регулярным выражением /\bjava\b/i. А чтобы найти все вхождения этого слова в строке, можно добавить флаг g: /\bjava\b/gi.

Методы класса String для поиска по шаблону

До этого момента мы обсуждали грамматику создаваемых регулярных выражений, но не рассматривали, как эти регулярные выражения могут фактически использоваться в JavaScript-сценариях. В данном разделе мы обсудим методы объекта String, в которых регулярные выражения применяются для поиска по шаблону, а также для поиска с заменой. А затем продолжим разговор о поиске по шаблону с регулярными выражениями, рассмотрев объект RegExp, его методы и свойства.

Строки поддерживают четыре метода, использующие регулярные выражения. Простейший из них - метод search(). Он принимает в качестве аргумента регулярное выражение и возвращает либо позицию первого символа найденной подстроки, либо -1, если соответствие не найдено. Например, следующий вызов вернет 4:

```
var result = "JavaScript".search(/script/i);    // 4
```

Если аргумент метода search() не является регулярным выражением, он сначала преобразуется путем передачи конструктору RegExp. Метод search() не поддерживает глобальный поиск и игнорирует флаг g в своем аргументе.

Метод replace() выполняет операцию поиска с заменой. Он принимает в качестве первого аргумента регулярное выражение, а в качестве второго - строку замены. Метод отыскивает в строке, для которой он вызван, соответствие указанному шаблону.

Если регулярное выражение содержит флаг g, метод replace() заменяет все найденные совпадения строкой замены. В противном случае он заменяет только первое найденное совпадение. Если первый аргумент метода replace() является строкой, а не регулярным выражением, то метод выполняет буквальный поиск строки, а не преобразует его в регулярное выражение с помощью конструктора RegExp(), как это делает метод search().

В качестве примера мы можем воспользоваться методом replace() для единообразной расстановки прописных букв в слове «JavaScript» для всей строки текста:

```
// Независимо от регистра символов заменяем словом в нужном регистре
var result = "javascript".replace(/JavaScript/ig, "JavaScript");
```

Метод replace().

Метод replace() представляет собой более мощное средство, чем можно было бы предположить по этому примеру. Напомню, что подвыражения в скобках, находящиеся внутри регулярного выражения, нумеруются слева направо, и что регулярное выражение запоминает текст, соответствующий каждому из подвыражений. Если в строке замены присутствует знак $ с цифрой, метод replace() заменяет эти два символа текстом, соответствующим указанному подвыражению. Это очень полезная возможность. Мы можем использовать ее, например, для замены прямых кавычек в строке типографскими кавычками, которые имитируются ASCII-символами:

```
// Цитата - это кавычка, за которой следует любое число символов,
// отличных от кавычек (их мы запоминаем), за этими символами
// следует еще одна кавычка
var quote = /"([^"]*)"/g;

// Заменяем прямые кавычки типографскими и оставляем без изменений «$1»
// содержимое цитаты, хранящееся в $1
var text = '"JavaScript" - интерпретируемый язык программирования.';
var result = text.replace(quote, "«$1»");      // «JavaScript» - интерпретируемый язык программирования.
```

Важное, что следует отметить, - второй аргумент replace() может быть функцией, динамически вычисляющей строку замены.

Метод match().

Метод match() - это наиболее общий из методов класса String, использующих регулярные выражения. Он принимает в качестве единственного аргумента регулярное выражение (или преобразует свой аргумент в регулярное выражение, передав его конструктору RegExp()) и возвращает массив, содержащий результаты поиска. Если в регулярном выражении установлен флаг g, метод возвращает массив всех соответствий, присутствующих в строке. Например,

```
// вернет ["1", "2", "3"]
var result = "1 плюс 2 равно 3".match(/\d+/g);
```

Если регулярное выражение не содержит флаг g, метод match() не выполняет глобальный поиск; он просто ищет первое совпадение. Однако match() возвращает массив, даже когда метод не выполняет глобальный поиск. В этом случае первый элемент массива - это найденная подстрока, а все оставшиеся элементы представляют собой подвыражения регулярного выражения. Поэтому если match() возвращает массив arr, то arr[0] будет содержать найденную строку целиком, arr[1] -подстроку, соответствующую первому подвыражению, и т.д. Проводя параллель с методом replace(), можно сказать, что в arr[n] заносится содержимое $n.

Например, взгляните на следующий программный код, выполняющий разбор URL-адреса:

```
var url = /(\w+):\/\/([\w.]+)\/(\S*)/;
var text = "Посетите наш сайт http://www.professorweb.ru/index.php";
```

```
var result = text.match(url);
if (result != null) {
        var fullurl = result[0];        // Содержит "http://www.professorweb.ru/index.php"
        var protocol = result[1];       // Содержит "http"
        var host = result[2];           // Содержит "www.professorweb.ru"
        var path = result[3];           // Содержит "index.php"
}
```

Следует отметить, что для регулярного выражения, в котором не установлен флаг g глобального поиска, метод match() возвращает то же значение, что и метод exec() регулярного выражения: возвращаемый массив имеет свойства index и input, как описывается в обсуждении метода exec() ниже.

Последний из методов объекта String, в котором используются регулярные выражения - split(). Этот метод разбивает строку, для которой он вызван, на массив подстрок, используя аргумент в качестве разделителя. Например,

```
"123,456,789".split(",");               // Вернет ["123","456","789"]
```

Метод split() может также принимать в качестве аргумента регулярное выражение. Это делает метод более мощным. Например, можно указать разделитель, допускающий произвольное число пробельных символов с обеих сторон:

```
"1, 2, 3 , 4 , 5".split(/\s*,\s*/);                 // Вернет ["1","2","3","4","5"]
```

Объект RegExp

Как было упомянуто, регулярные выражения представлены в виде объектов RegExp. Помимо конструктора RegExp(), объекты RegExp поддерживают три метода и несколько свойств.

Конструктор RegExp() принимает один или два строковых аргумента и создает новый объект RegExp. Первый аргумент конструктора - это строка, содержащая тело регулярного выражения, т.е. текст, который должен находиться между символами слэша в литерале регулярного выражения. Обратите внимание, что в строковых литералах и регулярных выражениях для обозначения управляющих последовательностей используется символ \, поэтому, передавая конструктору RegExp() регулярное выражение в виде строкового литерала, необходимо заменить каждый символ \ парой символов \\.

Второй аргумент RegExp() может отсутствовать. Если он указан, то определяет флаги регулярного выражения. Это должен быть один из символов g, i, m либо комбинация этих символов. Например,

```
// Находит все пятизначные числа в строке. Обратите внимание
// на использование в этом примере символов \\
var zipcode = new RegExp("\\d{5}", "g");
```

Конструктор RegExp() удобно использовать, когда регулярное выражение создается динамически и поэтому не может быть представлено с помощью синтаксиса литералов регулярных выражений. Например, чтобы найти строку, введенную пользователем, надо создать регулярное выражение во время выполнения с помощью RegExp().

Свойства RegExp.

Каждый объект RegExp имеет пять свойств. Свойство source - строка, доступная только для чтения, содержащая текст регулярного выражения. Свойство global - логическое значение, доступное только для чтения, определяющее наличие флага g в регулярном выражении. Свойство ignoreCase - это логическое значение, доступное только для чтения, определяющее наличие флага i в регулярном выражении. Свойство multiline - это логическое значение, доступное только для чтения, определяющее наличие флага m в регулярном выражении. И последнее свойство lastIndex - это целое число, доступное для чтения и записи. Для шаблонов с флагом g это свойство содержит номер позиции в строке, с которой должен быть начат следующий поиск. Как описано ниже, оно используется методами exec() и test().

Методы RegExp.

Объекты RegExp определяют два метода, выполняющие поиск по шаблону; они ведут себя аналогично методам класса String, описанным выше. Основной метод класса RegExp, используемый для поиска по шаблону - exec(). Он похож на упоминавшийся метод match() класса String, за исключением того, что является методом класса RegExp, принимающим в качестве аргумента строку, а не методом класса String, принимающим аргумент RegExp.

Метод exec() выполняет регулярное выражение для указанной строки, т.е. ищет совпадение в строке. Если совпадение не найдено, метод возвращает null. Однако если соответствие найдено, он возвращает такой же массив, как массив, возвращаемый методом match() для поиска без флага g. Нулевой элемент массива содержит строку, соответствующую регулярному выражению, а все последующие элементы - подстроки, соответствующие всем подвыражениям. Кроме того, свойство index содержит номер позиции символа, которым начинается соответствующий фрагмент, а свойство input ссылается на строку, в которой выполнялся поиск.

В отличие от match(), метод exec() возвращает массив, структура которого не зависит от наличия в регулярном выражении флага g. Напомню, что при передаче глобального регулярного выражения метод match() возвращает массив найденных соответствий. А exec() всегда возвращает одно соответствие, но предоставляет о нем полную информацию. Когда exec() вызывается для регулярного выражения, содержащего флаг g, метод устанавливает свойство lastIndex объекта регулярного выражения равным номеру позиции символа, следующего непосредственно за найденной подстрокой.

Когда метод exec() вызывается для того же регулярного выражения второй раз, он начинает поиск с символа, позиция которого указана в свойстве lastIndex. Если exec() не находит соответствия, свойство lastIndex получает значение 0. (Вы также можете установить lastIndex в ноль в любой момент, что следует делать во всех тех случаях, когда поиск завершается до того, как будет найдено последнее соответствие в одной строке, и начинается поиск в другой строке с тем же объектом RegExp.) Это особое поведение позволяет вызывать exec() повторно для перебора всех соответствий регулярному выражению в строке. Например,

```
var pattern = /Java/g;
var text = "JavaScript - это более забавная штука, чем Java!";
var result;
while((result = pattern.exec(text)) != null) {
        console.log("Найдено '" + result[0] + "'" +
                    " в позиции " + result.index +
                    "; следующий поиск начнется с " + pattern.lastIndex);
```

```
}
```

Еще один метод объекта RegExp - test(), который намного проще метода exec(). Он принимает строку и возвращает true, если строка соответствует регулярному выражению:

```
var pattern = /java/i;
pattern.test("JavaScript");          // Вернет true
```

Вызов test() эквивалентен вызову exec(), возвращающему true, если exec() возвращает не null. По этой причине метод test() ведет себя так же, как метод exec() при вызове для глобального регулярного выражения: он начинает искать указанную строку с позиции, заданной свойством lastIndex, и если находит соответствие, устанавливает свойство lastIndex равным номеру позиции символа, непосредственно следующего за найденным соответствием. Поэтому с помощью метода test() можно так же сформировать цикл обхода строки, как с помощью метода exec().

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

Печатные издания, посвященные изучению работы с регулярными выражениями:

1. Гойвертс Я., Левитан С. Регулярные выражения. Сборник рецептов. – Пер. с англ. – СПб.: Символ-Плюс, 2010. – 608 с.
2. Джеффри Фридл. Регулярные выражения. – Пер. с англ. – СПб.: Символ-Плюс, 2008. – 608 с.
3. Майкл Фицджеральд. Регулярные выражения. Основы. – Пер. с англ. – М.: Вильямс, 2015. – 400с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

4. Что такое регулярное выражение? Назначение и область использования.
5. Как определяется регулярное выражение в JavaScript?
6. Приведите примеры символов литералов в регулярных выражениях.
7. Для чего вводятся классы символов в регулярных выражениях?
8. Что такое повторение? Приведите примеры.
9. Как определяется альтернатива в регулярных выражениях?
10. Что позволяют делать группировки?
11. Какие флаги приняты в регулярных выражениях языка JavaScript?
12. Какие стандартные методы класса String можно использовать для организации поиска с использованием регулярных выражений?
13. Какие возможности предоставляет объект RegExp?